SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:	Abinaya Sri J	**Reg. No.:**	192524429
Date:	24/10/26	**Duration:**	60 minutes

Code of Conduct

I Abinaya Sri J (192524429) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the Student:

Annexure A – Architecture Design Assignment

1. Analyze System Requirements

1.1 Introduction

The Smart Healthcare Appointment and Patient Management System is a web-based application designed to simplify hospital appointment scheduling and patient information management. It provides a centralized platform for patients, doctors, receptionists, and hospital administrators to manage appointments, doctor availability, patient records, and reports.

The proposed system reduces manual paperwork, minimizes appointment scheduling conflicts, improves communication, and provides secure access to healthcare information.

1.2 System Objectives

The main objectives are:

1. To allow patients to register, log in, and book appointments online.
2. To enable doctors to manage their availability and view scheduled appointments.
3. To help receptionists manage appointments and patient registration.
4. To enable administrators to manage users, doctors, and system reports.
5. To send appointment confirmations, reminders, and cancellation notifications.
6. To maintain patient and appointment information securely.
7. To improve the efficiency, reliability, and accessibility of hospital services.

1.3 Functional Requirements

ID	Functional Requirement
FR-01	The system shall provide secure registration and login for authorized users.
FR-02	Patients shall be able to view available doctors and book appointments.
FR-03	Patients shall be able to view, reschedule, or cancel their appointments according to hospital policies.
FR-04	Doctors shall be able to manage their availability and view scheduled appointments.

FR-05	Receptionists shall be able to register patients and manage appointment bookings.
FR-06	Administrators shall be able to manage users, doctors, and system settings.
FR-07	The system shall maintain patient and appointment records.
FR-08	The system shall send appointment confirmations, reminders, and cancellation notifications.
FR-09	The system shall generate appointment and patient-management reports for authorized staff.
FR-10	The system shall maintain audit logs for important administrative activities.

1.4 Non-Functional Requirements

Requirement	Description
Scalability	The system should support increasing numbers of patients, doctors, and appointments.
Security	Authentication, role-based access control, encryption, and secure session management should protect sensitive information.
Performance	Common operations should respond quickly under the expected workload. A proposed target is within 2 seconds for 95% of routine requests under normal load.
Reliability	The system should prevent duplicate bookings and maintain consistent appointment records.
Availability	The system should target 99.5% availability per month, excluding planned maintenance, subject to infrastructure and operational support.
Maintainability	The system should use modular components, clear interfaces, documentation, and automated tests.
Usability	The interface should be simple, accessible, and usable on desktop and mobile devices.
Data Integrity	Database constraints and transactions should protect the accuracy and consistency of records.

1.5 Quality Attributes Influencing Architecture

The following quality attributes directly influence the architecture:

- **Scalability:** Additional application instances should be deployable when the number of users increases.
- **Maintainability:** Separating the presentation, business logic, and database layers makes changes easier.
- **Reliability:** Database transactions and validation prevent inconsistent or duplicate appointment records.
- **Availability:** Backups, health monitoring, and recovery procedures reduce service interruptions.
- **Performance:** Database indexing, pagination, and efficient queries reduce response times.
- **Security:** Authentication, authorization, encryption, and audit logging protect patient information.

2. Select a Suitable Software Architecture

2.1 Selected Architecture: Three-Tier Layered Client-Server Architecture

The proposed system uses a **three-tier layered client-server architecture with MVC-based separation within the presentation and application layers**.

The three main tiers are:

1. Presentation Layer
2. Application and Business Logic Layer
3. Data Layer

The architecture also integrates an external notification service for sending appointment messages.

2.2 Presentation Layer

This layer provides the user interface through a web browser or mobile-friendly web application. Patients, doctors, receptionists, and administrators access the functions permitted by their roles.

Responsibilities include:

- Displaying login and registration pages.
- Showing available doctors and appointment schedules.
- Providing appointment booking and cancellation forms.
- Displaying dashboards, notifications, and reports.
- Sending user requests to the application layer through secure APIs.

2.3 Application and Business Logic Layer

This layer processes requests and enforces the system's business rules.

It contains the following modules:

- **Authentication and Authorization Module:** Verifies user identity and permissions.
- **Patient Management Module:** Manages patient profiles and registration.
- **Doctor Management Module:** Maintains doctor information and availability.
- **Appointment Management Module:** Books, reschedules, cancels, and validates appointments.
- **Notification Module:** Requests appointment confirmations and reminders.
- **Reporting Module:** Generates appointment summaries and administrative reports.
- **Audit Module:** Records important activities for monitoring and accountability.

2.4 Data Layer

The data layer uses a relational database such as MySQL or PostgreSQL.

It stores:

- User accounts and roles.
- Patient profiles.
- Doctor profiles and availability.
- Appointment details and statuses.
- Consultation records, where authorized and required.
- Notification history.
- Audit logs.

Primary keys, foreign keys, unique constraints, indexes, and database transactions help maintain data integrity.

2.5 Justification for the Selected Architecture

The three-tier layered architecture is appropriate because the system has clearly separated user-interface, business-processing, and data-management responsibilities.

It offers the following benefits:

1. **Separation of concerns:** Changes to the interface do not require rewriting the entire application.
2. **Security:** Business rules and access checks are enforced on the server rather than relying on the browser.
3. **Maintainability:** Each module can be developed, tested, and maintained independently within the application.
4. **Scalability:** Additional application instances can be introduced behind a load balancer when required.
5. **Cost-effectiveness:** It is simpler to deploy and operate initially than a distributed microservices architecture.

6. **Integration:** External notification providers and future hospital systems can be connected through defined interfaces.

3. Draw the Software Architecture Diagram

3.1 Proposed Architecture Diagram

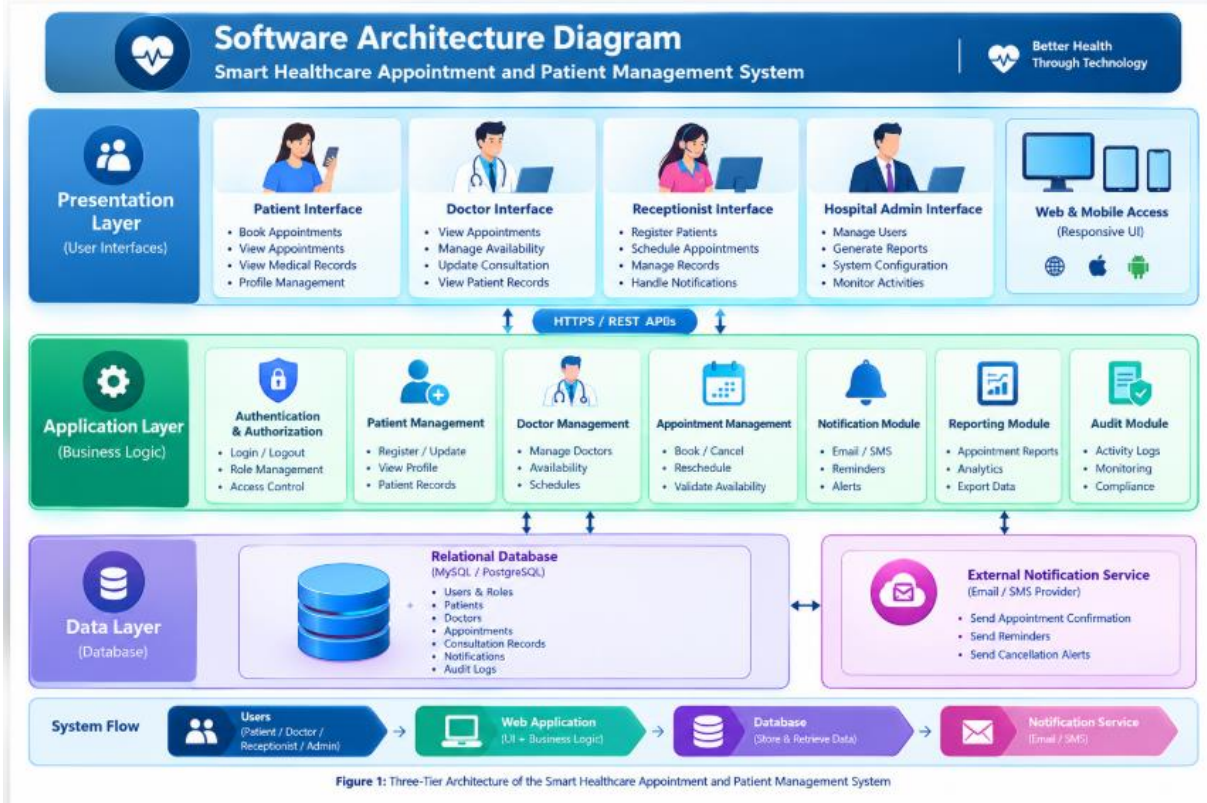


Figure 1: Three-Tier Architecture of the Smart Healthcare Appointment and Patient Management System

Communication mechanisms:

- Users access the system through HTTPS.
- The presentation layer communicates with the application layer through authenticated REST API requests.
- The application layer accesses the database through a secure database connection.
- The notification module communicates with the external email/SMS provider through its API.

- Database transactions and constraints maintain consistent appointment records.

4. Explain Components and Their Interactions

4.1 Component Responsibilities

Component	Responsibility
Web Interface	Accepts user input and displays information.
Authentication Module	Validates login credentials and manages authenticated sessions or tokens.
Authorization Module	Checks whether the user's role permits the requested operation.
Patient Management	Creates and updates authorized patient profiles.
Doctor Management	Maintains doctor profiles, departments, and availability schedules.
Appointment Management	Checks availability and processes booking, rescheduling, and cancellation.
Notification Module	Sends appointment-related messages through an external provider.
Reporting Module	Produces authorized appointment and administrative reports.
Audit Module	Records significant user and administrative actions.
Relational Database	Stores and retrieves structured application data.
External Notification Service	Delivers email or SMS messages requested by the application.

4.2 Component Interaction

The components interact through defined interfaces. The presentation layer sends requests to the application layer, which validates permissions and executes the relevant business rules. When data is needed, the application layer communicates with the database.

For example, when a patient books an appointment, the appointment management module checks the doctor's availability, validates the requested time, and attempts to create the booking within a database transaction. The system prevents conflicting bookings through appropriate transaction handling and database constraints.

After a successful booking, the notification module requests an appointment confirmation. If the external notification provider is temporarily unavailable, the system should retain the notification request for retry rather than losing the appointment record.

4.3 Overall System Workflow: Appointment Booking

1. The patient opens the web application and logs in.
2. The authentication module verifies the patient's identity.
3. The patient selects a doctor, date, and available time slot.
4. The appointment module validates the request and checks availability.
5. The database transaction creates the appointment while preventing conflicting bookings.
6. The application returns the booking status to the patient.
7. The notification module requests a confirmation through the external provider.
8. The doctor and authorized reception staff can view the updated appointment schedule.
9. The system records relevant activity in the audit log.

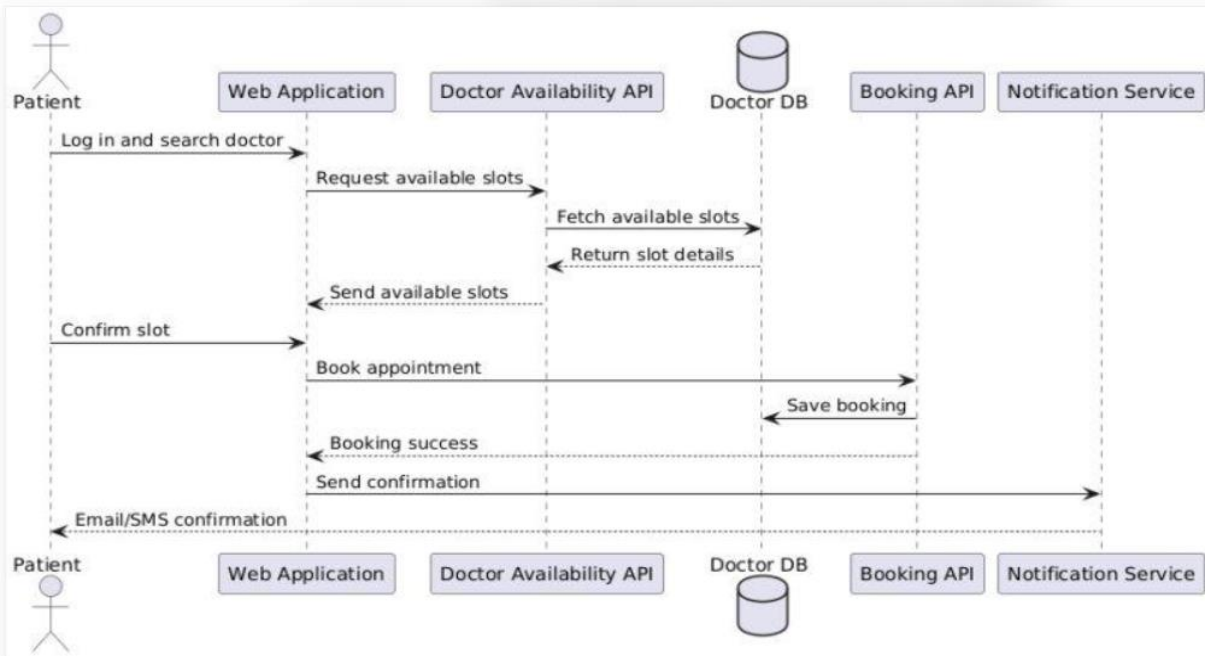


Figure 2: Appointment Booking Sequence Diagram

4.4 Workflow for Appointment Cancellation

1. The patient or authorized staff member requests cancellation.
2. The application verifies identity, role, appointment ownership, and cancellation rules.
3. The appointment module updates the appointment status in the database.
4. The system makes the time slot available again when appropriate.
5. A cancellation notification is requested.
6. The updated appointment information becomes available to authorized users.

5. Discuss Quality Attributes

5.1 Scalability

The architecture separates the user interface, application logic, and data storage. If the number of patients or appointments increases, the application layer can be scaled horizontally by deploying additional application instances behind a load balancer.

Database indexes, connection pooling, pagination, and query optimization help manage larger workloads. If future demand requires it, caching and read replicas can be introduced after evaluating consistency and performance requirements.

5.2 Security

Security is essential because the system stores personal and potentially sensitive healthcare information.

The proposed measures include:

- HTTPS encryption for communication.
- Secure password hashing rather than storing plain-text passwords.
- Role-based access control for patients, doctors, receptionists, and administrators.
- Server-side validation of all requests.
- Parameterized database queries to reduce SQL injection risks.
- Session expiration and secure session handling.
- Encryption of sensitive data where appropriate.
- Audit logging for important access and administrative operations.
- Regular security updates, backups, and access reviews.

The system should collect only necessary information and follow applicable privacy and healthcare data-protection requirements.

5.3 Performance

The architecture supports efficient processing through:

- Database indexing on appointment dates, doctor IDs, patient IDs, and relevant search fields.
- Efficient SQL queries and pagination for large result sets.

- Connection pooling to reduce repeated database connection overhead.
- Server-side validation before database operations.
- Asynchronous notification delivery so external email/SMS delays do not unnecessarily block booking confirmation.
- Performance monitoring and load testing to identify bottlenecks.

5.4 Reliability

Reliability is supported by database transactions, data validation, and constraints that prevent conflicting appointments. Error handling ensures that failed operations do not leave partially updated records.

Regular backups and tested restoration procedures protect against data loss. Failed notification attempts should be logged and retried without creating duplicate appointments or duplicate messages where idempotency can be applied.

5.5 Maintainability

The modular design separates authentication, patient management, doctor management, appointments, notifications, and reporting.

This improves maintainability because:

- Individual modules can be modified with limited impact on other modules.
- Clear interfaces reduce dependencies between components.
- Automated unit, integration, and regression tests help detect defects.
- Documentation supports future developers.
- Version control and controlled database migrations support safe updates.

5.6 Availability

The system should aim for a proposed monthly availability target of 99.5%, excluding planned maintenance.

Availability can be improved through:

- Health monitoring and application logging.
- Reliable hosting and database infrastructure.
- Automated backups and disaster-recovery procedures.
- Redundant application instances when the workload and budget justify them.
- Database recovery mechanisms and tested restoration plans.
- Alerting for application errors and infrastructure failures.

The architecture does not guarantee uninterrupted service by itself; actual availability depends on deployment, infrastructure, monitoring, and operational practices.

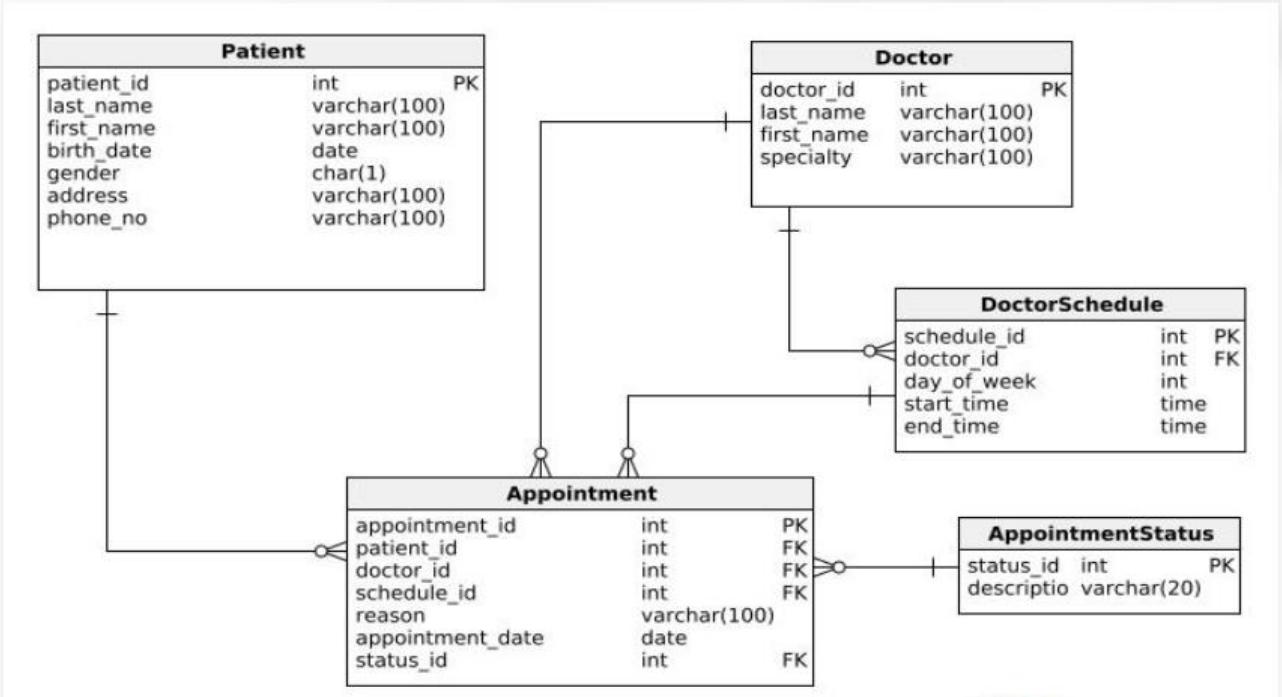


FIG 3: Entity Relationship Diagram (ERD)

6. Justify Architectural Decisions

6.1 Rationale Behind the Architectural Choices

The three-tier layered client-server architecture was selected because the system requires a clear separation between user interaction, business processing, and data storage.

The presentation layer manages interaction with different users. The application layer enforces appointment rules and access permissions. The data layer stores structured records and maintains consistency.

MVC-based separation helps organize interface-related responsibilities, while REST APIs provide defined communication interfaces between the presentation and application layers. A relational database is suitable because patients, doctors, appointments, and availability schedules have structured relationships.

The notification integration is separated from appointment processing so that external service failures do not invalidate successful bookings.

6.2 Architectural Trade-Offs

Decision	Advantages	Trade-Offs
Three-tier architecture	Clear separation of concerns and easier maintenance.	Requires well-defined interfaces between layers.
Modular application	Easier initial development and deployment.	Large modules may become harder to scale independently as the system grows.
Relational database	Supports transactions, relationships, and data integrity.	Requires careful query optimization as data volume increases.
REST API communication	Standardized integration and easier client support.	Adds network communication and API security requirements.
External notification provider	Reduces the need to build an email/SMS delivery system from scratch.	Introduces provider costs, delivery dependencies, and possible service outages.
Horizontal application scaling	Supports increased concurrent demand.	Requires load balancing, monitoring, and careful session management.

6.3 Support for Future Enhancements

The proposed architecture can support the following future improvements:

1. **Online Payments:** Integrate a secure payment gateway for consultation fees.
2. **Telemedicine:** Add video consultation services through an external communication provider.
3. **Mobile Application:** Reuse the existing APIs for a dedicated Android or iOS application.
4. **AI-Based Assistance:** Add appointment-demand forecasting or an appointment recommendation feature as a separate, controlled module.
5. **Hospital Integration:** Introduce APIs for integration with laboratory, pharmacy, or hospital information systems.
6. **Advanced Analytics:** Add dashboards for appointment trends, cancellations, and doctor workload.
7. **Cloud Deployment:** Deploy application instances and managed database services in a cloud environment.
8. **Microservices Migration:** If independent scaling, team ownership, or deployment needs justify the additional complexity, selected modules such as notifications or reporting can later be separated into services.

6.4 Long-Term Maintainability

Long-term maintainability will be supported by coding standards, modular design, API documentation, automated testing, secure configuration management, version control, and regular dependency updates.

Database migrations should be version-controlled, and changes to important business rules should be covered by automated tests. Monitoring and audit logs will help administrators diagnose problems and maintain accountability.

The architecture can therefore evolve as hospital requirements and user demand change without requiring the entire system to be redesigned.

Conclusion

The Smart Healthcare Appointment and Patient Management System adopts a three-tier layered client-server architecture with modular business components, REST API communication, a relational database, and an external notification service.

This architecture meets the major functional requirements while addressing scalability, security, performance, reliability, maintainability, and availability. Its clear separation of concerns simplifies development, testing, and future integration. Although the architecture introduces some operational overhead and dependencies, it offers a practical and cost-effective foundation for a healthcare appointment management application.

The proposed design provides a maintainable starting point that can be enhanced with online payments, telemedicine, mobile applications, analytics, and additional hospital integrations as future needs arise.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement Analysis (30%)	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.
Selection of Software Architecture (25%)	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or justification is missing.
Architecture Diagram and Component Design (20%)	Produces a clear, complete, and well-structured architecture diagram with all major components, interfaces, and interactions accurately represented.	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and lacks sufficient detail.	Diagram is incomplete, unclear, or technically incorrect.
Component Interaction and Quality Attribute Analysis (15%)	Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component interactions and quality attributes are poorly explained or missing.
Architectural Justification and Technical Presentation (10%)	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well-organized, professional report.	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of trade-offs or future scope; report needs improvement.	Justification is weak or absent; report lacks organization and technical clarity.

Criteria	**Mark**
System Requirement Analysis **(30%)**	/5
Selection of Software Architecture **(25%)**	/5
Architecture Diagram and Component Design **(20%)**	/5
Component Interaction and Quality Attribute Analysis **(15%)**	/5
Architectural Justification and Technical Presentation **(10%)**	/5
TOTAL	/25